

Prefix Sum

前缀和 & 差分

前缀和

定义

前缀和可以简单理解为「数列的前 项的和」，是一种重要的预处理方式，能大大降低查询的时间复杂度。¹

C++ 标准库中实现了前缀和函数 `std::partial_sum`，定义于头文件 `<numeric>` 中。

例题

- ▶ 例题
- ▶ 解题思路
- ▶ 参考代码

二维/多维前缀和

常见的多维前缀和的求解方法有两种。

基于容斥原理

这种方法多用于二维前缀和的情形。给定大小为 的二维数组，要求出其前缀和。那么，同样是大小为 的二维数组，且

类比一维的情形，应该可以基于 或 计算，从而避免重复计算前面若干项的和。但是，如果直接将 和 相加，再加上，会导致重复计算 这一重叠部分的前缀和，所以还需要再将这部分减掉。这就是 [容斥原理](#)。由此得到如下递推关系：

实现时，直接遍历 求和即可。

▼ 示例

考虑一个具体的例子。

$$A = \begin{pmatrix} 1 & 2 & 4 & 3 \\ 5 & 1 & 2 & 4 \\ 6 & 3 & 5 & 9 \end{pmatrix} \quad S = \begin{pmatrix} 1 & 3 & 7 & 10 \\ 6 & 9 & 15 & 22 \\ 12 & 18 & 29 & 45 \end{pmatrix}$$

这里，是给定矩阵 的前缀和。根据定义，是左图中虚线方框中的子矩阵的和。这里，是蓝色子矩阵的和，是红色子矩阵的和，它们重叠部分的和是。由此可见，如果直接相加 和，会重复计算，所以应该有

同样的道理，在已经预处理出二位前缀和后，要查询左上角为、右下角为 的子矩阵的和，可以计算

这可以在 $O(n)$ 时间内完成。

在二维的情形，以上算法的时间复杂度可以简单认为是 $O(n^2)$ ，即与给定数组的大小成线性关系。但是，当维度 d 增大时，由于容斥原理涉及的项数以指数级的速度增长，时间复杂度会成为 $O(2^d \cdot n)$ ，这里 d 是数组维度，而 n 是给定数组大小。因此，该算法不再适用。

▼ [洛谷 P1387 最大正方形](#)

在一个 $n \times m$ 的只包含 0 和 1 的矩阵里找出一个不包含 0 的最大正方形，输出边长。

► 参考代码

逐维前缀和

对于一般的情形，给定 d 维数组，大小为 $n_1 \times n_2 \times \dots \times n_d$ ，同样要求其前缀和。这里，

从上式可以看出， d 维前缀和就等于 d 次求和。所以，一个显然的算法是，每次只考虑一个维度，固定所有其它维度，然后求若干个一维前缀和，这样对所有 d 个维度分别求和之后，得到的就是 d 维前缀和。

► 三维前缀和的参考实现

因为考虑每一个维度的时候，都只遍历了整个数组一遍，这样的算法复杂度是 $O(n^3)$ 的，通常可以接受。

特例：子集和 DP

维度比较大的情形，经常出现在一类叫做 **子集和 (SOS, Sum Over Subsets)** 的问题中。这是高维前缀和的特例。

问题描述如下。考虑大小为 2^d 的集合的全体子集上面定义的函数 f ，现在要求出其子集和函数 F ，它满足

即 F 等于其所有子集 S 上的函数值 $f(S)$ 的和。

首先，子集和问题可以写成高维前缀和的形式。注意到，子集 S 可以通过状态压缩的思想表示为长度为 d 的 0-1 字符串。将字符串的每一位都看作是数组下标的一个维度，那么 S 其实就是一个 d 维数组，且每个维度下标都一定在 $[0, 1]$ 之间。同时，子集的包含关系就等价于下标的大小关系，即

所以，对子集求和，就是求这个 d 维数组的前缀和。

现在，可以直接使用前文所述的逐维前缀和的方法求得子集和。时间复杂度是 $O(d \cdot 2^d)$ 。

► 参考实现

子集和的逆操作需要通过 [容斥原理](#) 进行。子集和问题也是快速莫比乌斯变换的必要步骤之一。

树上前缀和

设 sum_u 表示结点 u 到根节点的权值总和。

然后：

- 若是点权，路径上的和为 sum_u 。
- 若是边权，路径上的和为 $sum_u - sum_{fa_u}$ 。

LCA 的求法参见 [最近公共祖先](#)。

差分

解释

差分是一种和前缀和相对的策略，可以当做是求和的逆运算。

这种策略的定义是令

性质

- 的值是 的前缀和，即
- 计算 的前缀和

它可以维护多次对序列的一个区间加上一个数，并在最后询问某一位的数或是多次询问某一位的数。注意修改操作一定要在查询操作之前。

▼ 示例

譬如使 中的每个数加上一个，即

其中，

最后做一遍前缀和就好了。

C++ 标准库中实现了差分函数 [std::adjacent_difference](#)，定义于头文件 `<Ltnumeric>` 中。

树上差分

树上差分可以理解为对树上的某一段路径进行差分操作，这里的路径可以类比一维数组的区间进行理解。例如在对树上的一些路径进行频繁操作，并且询问某条边或者某个点在经过操作后的值的时候，就可以运用树上差分思想了。

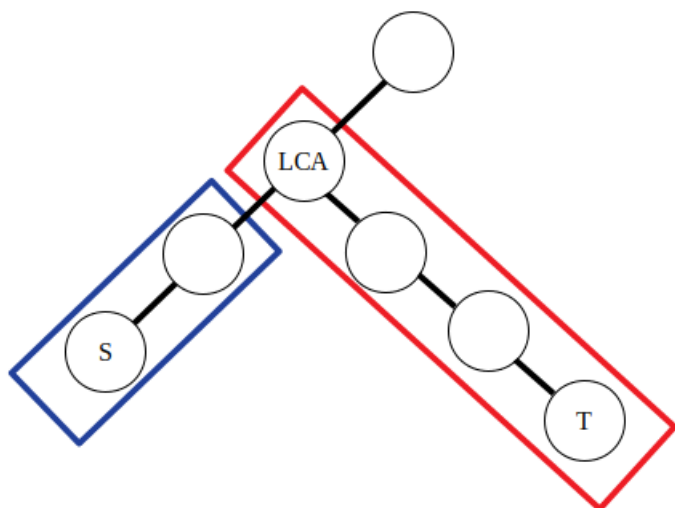
树上差分通常会结合 [树基础](#) 和 [最近公共祖先](#) 来进行考察。树上差分又分为 **点差分** 与 **边差分**，在实现上会稍有不同。

点差分

举例：对树上的一些路径 进行访问，问一条路径 上的点被访问的次数。

对于一次 的访问，需要找到 与 的公共祖先，然后对这条路径上的点进行访问（点的权值加一），若采用 DFS 算法对每个点进行访问，由于有太多的路径需要访问，时间上承受不了。这里进行差分操作：

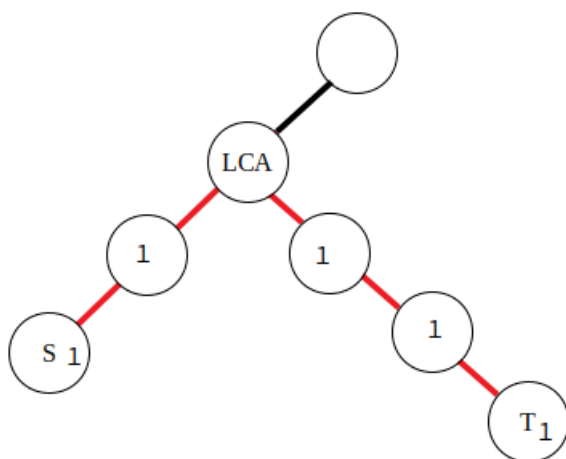
其中 表示 的父亲节点， 为点权 的差分数组。



可以认为公式中的前两条是对蓝色方框内的路径进行操作，后两条是对红色方框内的路径进行操作。不妨令左侧的直系子节点为 s 。那么有 $s, s+1, \dots, LCA$ 。可以发现实际上点差分的操作和上文一维数组的差分操作是类似的。

边差分

若是对路径中的边进行访问，就需要采用边差分策略了，使用以下公式：



由于在边上直接进行差分比较困难，所以将本来应当累加到红色边上的值向下移动到附近的点里，那么操作起来也就方便了。对于公式，有了点差分的理解基础后也不难推导，同样是对两段区间进行差分。

例题

▼ [洛谷 3128 最大流](#)

FJ 给他的牛棚的 n 个隔间之间安装了 m 根管道，隔间编号从 1 到 n 。所有隔间都被管道连通了。

FJ 有 k 条运输牛奶的路线，第 i 条路线从隔间 s_i 运输到隔间 t_i 。一条运输路线会给它的两个端点处的隔间以及中间途径的所有隔间带来一个单位的运输压力，你需要计算压力最大的隔间的压力是多少。

- 解题思路
- 参考代码

习题

前缀和：

- [洛谷 B3612 【深进 1. 例 1】 求区间和](#)
 - [洛谷 U69096 前缀和的逆](#)
 - [AtCoder joi2007ho_a 最大の和](#)
 - [「USACO16JAN」 子共七 Subsequences Summing to Sevens](#)
 - [「USACO05JAN」 Moo Volume S](#)
-

二维/多维前缀和：

- [HDU 6514 Monitor](#)
 - [洛谷 P1387 最大正方形](#)
 - [「HNOI2003」 激光炸弹](#)
 - [CF 165E Compatible Numbers](#)
 - [CF 383E Vowels](#)
 - [ARC 100C Or Plus Max](#)
-

树上前缀和：

- [LOJ 10134.Dis](#)
 - [LOJ 2491. 求和](#)
-

差分：

- [树状数组 3： 区间修改， 区间查询](#)
 - [P3397 地毯](#)
 - [「Poetize6」 IncDec Sequence](#)
-

树上差分：

- [洛谷 3128 最大流](#)
 - [JLOI2014 松鼠的新家](#)
 - [NOIP2015 运输计划](#)
 - [NOIP2016 天天爱跑步](#)
-

参考资料与注释

1. 南海区青少年信息学奥林匹克内部训练教材 [↩](#)
-

本页面最近更新：2025/4/9 00:35:08, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [ChungZH](#), [H-J-Granger](#), [NachtgeistW](#), [Alpacabla](#), [Alpha1022](#), [Back1ght](#), [c-forrest](#),

[Chrogeek](#), [countercurrent-time](#), [diaoweb](#), [Enter-tainer](#), [Henry-ZHR](#), [iamtwz](#), [lr1d](#), [Junyan721113](#), [kenlig](#), [Kensuke-Hinata](#), [ksyx](#), [LeiJinpeng](#), [LeoJacob](#), [leoleoasd](#), [mgt](#), [ouuan](#), [Planet6174](#), [ShaoChenHeng](#), [shawlleyw](#), [sshwy](#), [SukkaW](#), [Tiphereth-A](#), [wuyudi](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用